

TCC805x MCU BSP

User Guide

Rev. 1.02 [A]
2021-04-09

※ The information in this document is subject to change without notice and should not be construed as a commitment by Telechips Inc.

Kindly visit <http://www.telechips.com> for more information.

© 2021 Telechips Inc. All rights reserved.

TABLE OF CONTENTS

Contents

1	Introduction	3
2	Terms and Abbreviations	3
3	Software Architecture	4
3.1	Device Driver	4
3.2	Operating System (OS)	5
3.3	System Adaptation Layer (SAL)	5
3.4	Sample Application	5
4	Directory Structure	6
5	Bootup Process	7
6	Reset Process	9
6.1	AP System Reset	9
6.2	Cold Reset	11
7	Development Environment	12
8	Memory Map	13
8.1	Load View	14
8.2	Execution View	14
8.3	Define Memory Region Base Address	14
8.4	Relocate RW and ZI	16
8.5	Define Memory Attribute for Each Area using MPU	19
8.6	Use Fixed DMA Buffer	20
8.7	Detailed View of SRAM	22
9	MCU BSP RTOS	23
9.1	Initiation Procedure	24
9.2	SAL Interface	24
9.3	FreeRTOS OS Kernel	24
9.3.1	FreeRTOS Directory Structure	24
9.3.1.1	OS	25
9.3.1.2	SAL	25
9.3.2	Configuration for FreeRTOS	25
9.3.3	Configuration for Task Priorities	26
9.3.4	Customizing "FreeRTOSConfig.h" File	27
9.3.5	Memory Resource Function	27
9.3.6	Exception Handler	28
10	Use Technology by the Third Party Within MCU BSP	29
11	References	30
12	Revision History	31
	Rev. 1.02: 2021-04-09	31
	Rev. 1.01: 2021-01-22	31
	Rev. 1.00: 2020-11-12	31

Figures

Figure 3.1	Block Diagram of Software Components in TCC805x MCU BSP	4
Figure 4.1	Directory Structure of TCC805x MCU BSP	6
Figure 5.1	MCU Boot Sequence	7
Figure 5.2	MCU Bootup Process	8
Figure 8.1	MCU BSP Memory Map	13
Figure 8.2	Predefinition for DMA (Green Hills IDE Environment)	20
Figure 8.3	SRAM Map	22
Figure 9.1	RTOS in MCU BSP	23
Figure 9.2	Directory Structure of FreeRTOS	24
Figure 9.3	Green Hills Project for FreeRTOS	25

Tables

Table 2.1	Terms and Abbreviation	3
-----------	------------------------------	---

1 INTRODUCTION

This document describes TCC805x MCU BSP. The MCU BSP means all contents including source code, build environment, and tools you downloaded via Git.

2 TERMS AND ABBREVIATIONS

Table 2.1 Terms and Abbreviation

MCU	Micro Controller Unit. Cortex-R5 processor in TCC805x
BSP	Board Support Package
OS	Operating System
RTOS	Real Time OS
SAL	System Adaptation Layer
S-NOR	Serial NOR Flash device
BL	Boot Loader
F/W	Firmware
MPU	Memory Protection Unit
EVB	Evaluation Board
XIP	eXecute In Place

3 SOFTWARE ARCHITECTURE

Figure 3.1 shows software components in TCC805x MCU BSP. The TCC805x MCU BSP is composed of device driver, Real Time Operating System (RTOS), System Adaptation Layer (SAL), and sample application. The MCU BSP is categorized into three types (Sample, 3rd party, TC Support) by usage. The sample is a sample code for testing device driver. You can refer to the sample code while implementing your own solution, but there is no guarantee for quality. The MCU BSP has the 3rd party software such as RTOS, but this MCU BSP does not provide technical support for the RTOS itself. The device driver and SAL interface are supported by Telechips (TC Support).

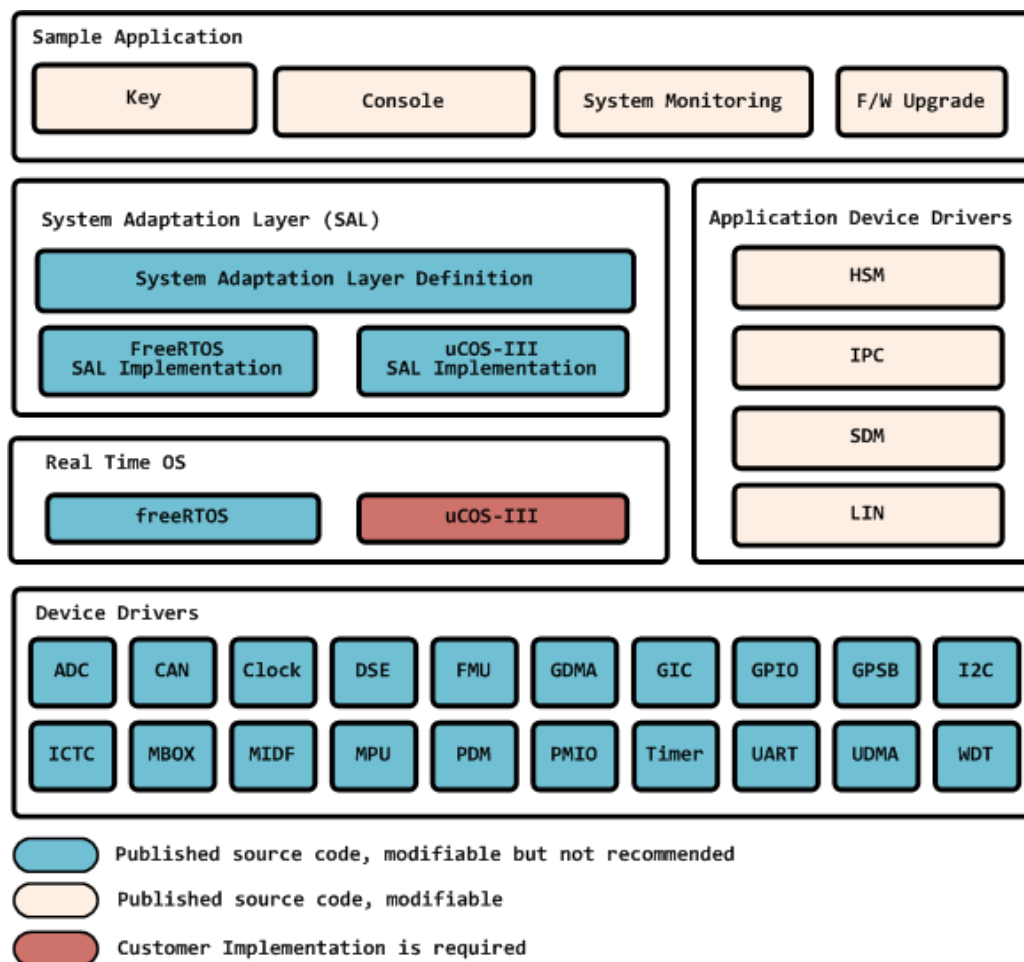


Figure 3.1 Block Diagram of Software Components in TCC805x MCU BSP

3.1 Device Driver

Certain device drivers in the MCU BSP support MCU function. Complex device driver such as CAN device driver can be added and developed by the third party or Tier 1, other than Telechips. For more detailed information on each device driver, refer to "*TCC805x MCU BSP-API Specification for xxx*". [2]

3.2 Operating System (OS)

The MCU BSP basically uses FreeRTOS which can be replaced by your own OS. If you have source code and license of uC/OS-III, you can easily replace it.

Note 1: The uC/OS-III source code is not included in the MCU BSP, but some portable components are included. The MCU BSP is optimized for uC/OS-III v4.04.03 and FreeRTOS v10.3.1. The uC/OS-III is supported based on the contract with Micrium. For more detailed information on uC/OS-III, refer to Micrium's website. [4]

Note 2: The FreeRTOS is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. Full license text is available on the following link; <http://www.freertos.org/a00114.html>

3.3 System Adaptation Layer (SAL)

SAL is an adaptation layer to change OS component and system resources. For detailed information on SAL, refer to "*TCC805x MCU BSP-API Specification for System Adaptation Layer*". [2]

3.4 Sample Application

The TCC805x MCU BSP provides the sample application for the Telechips EVB applied with the sample code. According to the OEM specifications, Tier 1 should develop its own application. The existing solution can be also applied.

4 DIRECTORY STRUCTURE

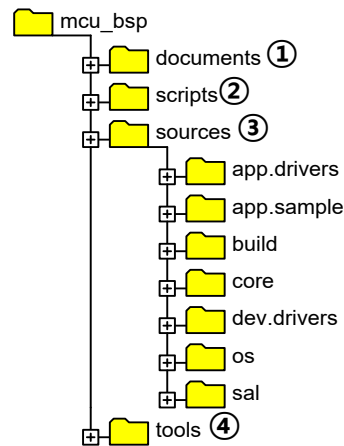


Figure 4.1 Directory Structure of TCC805x MCU BSP

- ① documents: All of documents for TCC805x MCU BSP
- ② scripts: Debug script for JTAG debugger such as Trace32
- ③ sources: Source code
 - A. app.drivers: Application drivers. These drivers do not access hardware directly.
 - B. app.sample: Sample applications
 - C. build: Build environment & utilities
 - D. core: Assembly core files
 - E. dev.drivers: Device drivers
 - F. os: Operating System
 - G. sal: System Adaptation Layer
- ④ tools: Tools for FWDN & F/W upgrade

5 BOOTUP PROCESS

Four elements are required to initialize MCU; TCC805x MCU Boot Code (MCU-BL0), External Boot Code (MCU-BL1), MCU firmware, and HSM Boot Code (HSM F/W).

Figure 5.1 shows the simple process. The MCU-BL0 is stored in the internal boot ROM and executed on internal SRAM when the power is turned on. The MCU-BL1, MCU main firmware, and HSM F/W are stored in the external S-NOR flash device. MCU-BL0 loads HSM F/W. And HSM F/W verifies MCU-BL1. MCU-BL1 is copied from S-NOR to SRAM and it is executed on SRAM. The MCU-BL1 executes MCU F/W. MCU F/W is executed on S-NOR as XIP. This document does not cover HSM F/W. For more information on HSM F/W, contact Telechips. [2]

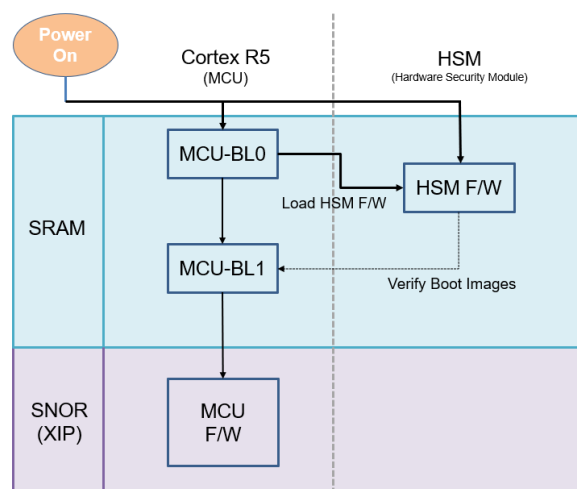


Figure 5.1 MCU Boot Sequence

Figure 5.2 shows detailed boot sequence.

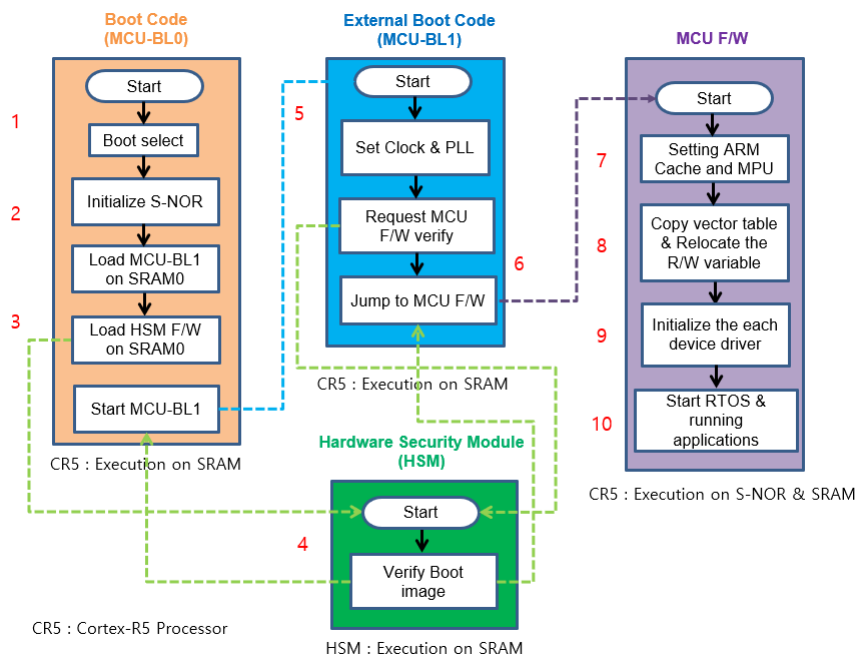


Figure 5.2 MCU Bootup Process

1. Boot TCC805x and load external boot code (MCU-BL0):
 - 1) Boot the chip in S-NOR boot mode. ----- (1)
 - 2) Initialize storage S-NOR to load MCU-BL1 code on SRAM. ----- (2)
 - 3) Load HSM F/W and request MCU-BL1 image verification. ----- (3)
2. Verify and execute MCU-BL1:
 - 1) Verify MCU-BL1 image. ----- (4)
 - 2) Execute MCU-BL1 and set MCU clock & PLL. ----- (5)
3. Verify and execute MCU F/W:
 - 1) Initialize Cortex-R5 core elements such as cache, MPU, ARM stack, etc. - startup.s ----- (7)
 - 2) Copy the vector table and relocate the R/W (.DATA) variable and initialize the Zero Initialize (.ZI) area. ----- (8)
 - 3) Initialize each device driver. ----- (9)
 - 4) Start the RTOS and run the application. ----- (10)

6 RESET PROCESS

TCC805x MCU BSP can reset Application Processor (AP) system or reset by itself.

6.1 AP System Reset

TCC805x MCU BSP resets AP system when Storage Core (SC) watchdog reset signal is generated.

AP system reset can be replaced to cold reset according to your scenario.

The following is a sequence of AP system reset.

1. Wait for any data to be entered into Operating System (OS) queue in MCU BSP.
 - 1) Refer to 'PMU_ResetSCWDTask()' function.
 - 2) This function is blocked by "SAL_QueueGet()" function until any data is entered into the OS queue.

mcu_bsp/sources/dev.drivers/pmu/pmu_reset.c

```
static void PMU_ResetSCWDTask
(
    void *                pArg
)
{
    ...
    while( 1 )
    {
        /* Wait for WDT IRQ from Storage Core */
        ( void ) SAL_QueueGet( gSCWdtControlId,
            ( void * ) &uiData,
            ( uint32 * ) &uiMsgSize,
            0,
            SAL_OPT_BLOCKING );
        ...
    }
}
```

2. SC watchdog signal is generated in SC.
3. SC watchdog interrupt is executed in MCU BSP.
 - 1) Refer to 'PMU_ResetSCWDIRQHandler()' function.
 - 2) This function enters data into OS queue by "SAL_QueuePut()".

mcu_bsp/sources/dev.drivers/pmu/pmu_reset.c

```
static void PMU_ResetSCWDTIQHandler
(
    void *                pArg
)
{
    ...
    while( 1 )
    {
        ( void ) SAL_QueuePut( gSCWdtControlId,
                               ( void * ) &uiData,
                               sizeof( uint32 ),
                               0,
                               SAL_OPT_BLOCKING );
    }
}
```

4. Reset AP system

- 1) After blocked function is released, MCU BSP resets AP system by calling “*PMU_ResetAPSystemReboot()*” function in “*PMU_ResetSCWDTTask()*” function.
- 2) Refer to the following function.

mcu_bsp/sources/dev.drivers/pmu/pmu_reset.c

```
static void PMU_ResetSCWDTTask
(
    void *                pArg
)
{
    ...
    PMU_ResetAPSystemReboot();
    ...
}
```

6.2 Cold Reset

TCC805x MCU BSP can perform a cold reset. Refer to the following function.

You can call this function in your application.

```
mcu_bsp/sources/dev.drivers/pmu/pmu_reset.c
```

```
void PMU_ResetColdReset  
(  
    void  
)  
{  
    SAL_WriteReg( ( uint32 ) 0x1, PMU_RST_REG_PMU_CRST ); //Write any value  
}
```

7 DEVELOPMENT ENVIRONMENT

This chapter is described in “*TC805x MCU BSP-Getting Stared*”. If you want to know about MCU BSP development environment, refer to “*Chapter 4. Download MCU BSP*” and “*Chapter 5. Build Guide*” in “*TC805x MCU-BSP Getting Stared*”. [2]

8 MEMORY MAP

In the TCC805x EVB environment, the memories available for the MCU are assumed to be exclusive to S-NOR memory for code operation and internal SRAM (512 KB). The memory layout changes by target product and by requirements.

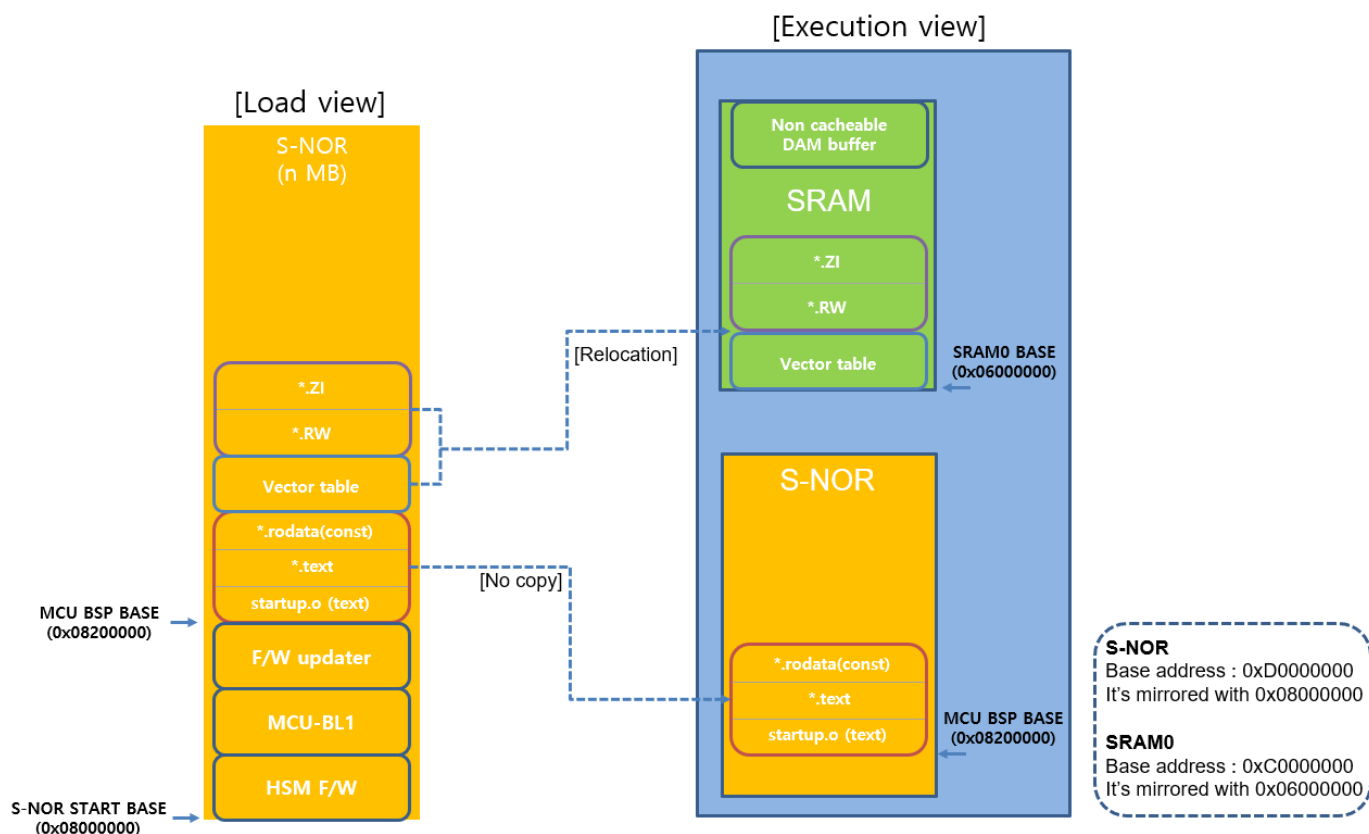


Figure 8.1 MCU BSP Memory Map

8.1 Load View

Load view in the Figure 8.1 above schematizes where the firmware image is written in the memory.

Target firmware image is generated after compile, and the linking process is saved in the S-NOR memory. Each area is classified as .text, .rodata, .rw, and .zi based on the property.

8.2 Execution View

Execution view in the Figure 8.1 above shows the status of memory used when the system operates after booting.

Since the S-NOR can execute the instruction at once, it does not need to copy data (.text, .rodata) to the separate RAM. However, others except = .text and .rodata should be copied to the RAM because the data (.RW, .ZI) must be able to read and write. But actually .ZI data does not need to be copied because value of .ZI should be 0 (zero). Therefore, .ZI is initialized to 0 instead of being copied.

That is, the code is executed in the S-NOR and the data is executed in the RAM. It is possible to remap the internal SRAM (512 KB) (0x06000000) to ARM vector address for the Cortex-R5 core, and the vector table must be located on the logical zero. Therefore, you should copy the vector table code to the internal SRAM.

8.3 Define Memory Region Base Address

In case of Green Hills IDE environment, assignment of load base address and data copy base address (SRAM area) can be adjusted in the linker directives file (.ld) of Green Hills IDE. The load base address is defined as "flash_memory", while the data copy base address is defined as "dram_memory".

■ Define Memory Layout (Green Hills IDE Environment)

```
mcu_bsp/sources/build/tcc805x/ghc/tgt/standalone_romcopy_tcc805x_snor_es.ld

MEMORY
{
    // Reserve 0x8000 bytes for use by target-resident code or debug monitor
    dram_memory : ORIGIN = 0x06000000, LENGTH = 0x80000-0x680-DMA_NC_SIZE
    // Reserve 0x680 bytes for arm stack
    stack_memory : ORIGIN = 0x06000000 + 0x80000 -DMA_NC_SIZE -0x680, LENGTH = 0x680
    // Reserve DMA_NC_SIZE bytes for non-cacheable area,
    // The dma address is valid only physical address(0xC0000000),
    // it can not use mirrored mem address(0x06000000)
    dma_memory : ORIGIN = 0xC0000000 + 0x80000-DMA_NC_SIZE, LENGTH = DMA_NC_SIZE
    // S-NOR code area XIP
    flash_memory : ORIGIN = 0x08200000, LENGTH = 0x00200000 //64MByte
}
```

In case of GCC environment, the memory layout is defined by project's linker script file (linker.ld) as below.

The load base address is defined as "NOR", while the data copy base address is defined as "SRAM".

■ Define Memory Layout (GCC Environment)

mcu_bsp/sources/build/tcc805x/gcc/Linker.Ld

MEMORY

```
{
    /*=====
    Region for Executing at SRAM with cacheable: 256KB
    =====*/
    SRAM (rwx) : ORIGIN = 0x06000000, LENGTH = (0x80000 - 0x680 - DMA_NC_SIZE)
    /* total(0x80000) - SRAM_ARM_STACK(0x680) - NC_SRAM(0x2000 or...config) */
    /*=====
    Region for ARM STACK area: 0x680Byte
    =====*/
    SRAM_ARM_STACK (rwx) : ORIGIN = 0x06000000 + (0x80000 - 0x680 - DMA_NC_SIZE), LENGTH = 0x680
    /*=====
    Region for Executing at SRAM with non-cacheable: 8KB,
    The dma address is valid only physical address(0xC0000000),
    it can not use mirrored mem address(0x06000000)
    =====*/
    NC_SRAM (rwx) : ORIGIN = 0xC0000000+0x80000-DMA_NC_SIZE, LENGTH = DMA_NC_SIZE
    /*=====
    Region for Executing at NOR Flash with cacheable
    =====*/
    NOR (rx) : ORIGIN = 0x08200000, LENGTH = 0x00200000
}
```


8.4 Relocate RW and ZI

Copying RW and initializing ZI area are executed in the *InitCVar()* function of *startup.s*. In the *InitCVar()* function, the location and size of each area, which are referred for initiation of the RW and ZI area, are based on variables defined by linker directives file (.ld) function of Green Hills IDE. The name, location, and size of variables can change arbitrarily depending on the use-cases. Refer to the following linker directives file.

■ Variable Definition of Each Area (Green Hills Environment)

mcu_bsp/sources/build/tcc805x/ghc/tgt/standalone_romcopy_tcc805x_snor_es.ld

```
// These special symbols mark the bounds of RAM and ROM memory.
__ghs_romstart    = MEMADDR(flash_memory);
__ghs_romend      = ADDR(.ROM.data);
__ghs_ramstart    = ADDR(.data);
__ghs_ramend      = ENDADDR(.bss);
__ghs_bssstart    = ADDR(.bss);
__arm_stack       = MEMADDR(stack_memory);
__nc_dmastart     = MEMADDR(dma_memory);
```

■ Variable Definition of Each Area (GCC Environment)

mcu_bsp/sources/build/tcc805x/gcc/Linker.Ld

```
/* .vector section which is used for vector table */
.vector:
{
    __VECTOR_START_LOAD = LOADADDR (.vector);
    __VECTOR_START__ = .;
    *vector.o (.vector .text .rodata .rodata*)
    . = ALIGN(0x20);
    __VECTOR_END__ = .;
} > SRAM AT > NOR
/* .data section which is used for initialized data */
.data:
    AT ( LOADADDR (.vector) + SIZEOF (.vector) )
{
    __DATA_START_LOAD = LOADADDR (.data);
    __DATA_START__ = .;
    *(.data)
    *(.data.*)
    . = ALIGN(0x20);
    __DATA_END__ = .;
} > SRAM
/* .bss section which is used for uninitialized data */
.bss (NOLOAD):
{
    . = ALIGN(0x20);
    __BSS_START__ = .;
    *(.bss)
    *(.bss.*)
    . = ALIGN(0x20);
    __BSS_END__ = .;
} > SRAM
```

The RW and ZI variables are manually relocated to main RAM of system by software while the system starts up.

■ Variable Reference of Each Area (Green Hills Environment)

mcu_bsp/sources/core/GHC/startup.asm

InitCVar

```
ldr r0, =__ghs_romend
ldr r1, =__ghs_ramstart
ldr r2, =__ghs_bssstart
```

```
cmp r0, r1
beq 5f
```

4: ; Copy RW Region

```
ldm r0!, {r4-r11}
stm r1!, {r4-r11}
cmp r1, r2
bne 4b
```

5: ; Clear BSS Region

```
ldr r0, =__ghs_bssstart
ldr r1, =__ghs_ramend
add r1, r1, #0x1F
bic r1, r1, #0x1F
cmp r0, r1
moveq pc, lr
```

```
mov r4, #0
mov r5, #0
mov r6, #0
mov r7, #0
mov r8, #0
mov r9, #0
mov r10, #0
mov r11, #0
```

6:

```
stm r0!, {r4-r11}
cmp r0, r1
bne 6b
```

```
mov pc, lr
```

■ Variable Reference of Each Area (GCC Environment)

mcu_bsp/sources/core/GCC/startup.S

InitCVar:

```
Ldr r0, =(__DATA_START_LOAD) @ Load address
Ldr r1, =(__DATA_START__)
Ldr r2, =(__DATA_END__)
```

```
cmp r0, r1
beq 5f
```

4: / Copy RW Region */*

```
ldm r0!, {r4-r11}
stm r1!, {r4-r11}
cmp r1, r2
bne 4b
```

5: / Clear BSS Region */*

```
Ldr r0, =(__BSS_START__)
Ldr r1, =(__BSS_END__)
cmp r0, r1
moveq pc, lr
```

```
mov r4, #0
mov r5, #0
mov r6, #0
mov r7, #0
mov r8, #0
mov r9, #0
mov r10, #0
mov r11, #0
```

6:

```
stm r0!, {r4-r11}
cmp r0, r1
bne 6b
```

```
mov pc, lr
```

8.5 Define Memory Attribute for Each Area using MPU

Cortex-R5 processor supports Memory Protection Unit (MPU).

The TCC805x MCU BSP sets the MPU as follows:

■ MPU Table for Setting Region

mcu_bsp/sources/dev.drivers/mpu/tcc805x/mpu.c

```
MPUConfig_t sMPUTable[ MPU_MAX_REGION ] =
{
    { MPU_REGION_ENABLE, MPU_BOOT_ADDR,      MPU_REGION_4GB,      MPU_EXECUTE_NEVER | MPU_PRIV_RW_USER_NA | MPU_STRONG_ORDERED_SHARED },
    { MPU_REGION_ENABLE, MPU_BOOT_ADDR,      MPU_REGION_256MB,    MPU_NORM_SHARED_WB_WA | MPU_PRIV_RW_USER_RW },
    { MPU_REGION_ENABLE, MPU_PERI_ADDR,       MPU_REGION_16MB,     MPU_STRONG_ORDERED_SHARED | MPU_PRIV_RW_USER_RW },
    { MPU_REGION_ENABLE, MPU_SRAM0_ADDR,      MPU_REGION_512KB,    MPU_NORM_SHARED_WB_WA | MPU_PRIV_RW_USER_RW },
    { MPU_REGION_ENABLE, MPU_SRAM1_ADDR,      MPU_REGION_64KB,     MPU_NORM_SHARED_WB_WA | MPU_PRIV_RW_USER_RW },
    { MPU_REGION_ENABLE, MPU_DMA_ADDR,        MPU_REGION_DMA,      MPU_STRONG_ORDERED_SHARED | MPU_PRIV_RW_USER_RW },
    { MPU_REGION_ENABLE, MPU_SNOR_ADDR,       MPU_REGION_128MB,    MPU_NORM_NSHARED_WB_NWA | MPU_PRIV_RO_USER_RO },
    { MPU_REGION_ENABLE, MPU_VECTOR_ADDR,     MPU_REGION_1KB,      MPU_NORM_SHARED_WB_WA | MPU_PRIV_RW_USER_RW },
    { MPU_REGION_ENABLE, MPU_A53_BOOT_ADDR,   MPU_REGION_256MB,    MPU_NORM_SHARED_WB_WA | MPU_PRIV_RW_USER_RW },
    { MPU_REGION_ENABLE, MPU_A53_BACKUP_ADDR, MPU_REGION_256MB,    MPU_NORM_SHARED_WB_WA | MPU_PRIV_RO_USER_RO },
    { MPU_REGION_ENABLE, MPU_CBUS_ADDR,       MPU_REGION_4MB,      MPU_STRONG_ORDERED_SHARED | MPU_PRIV_RW_USER_RW },
    { MPU_REGION_ENABLE, MPU_VIOC_ADDR,       MPU_REGION_4MB,      MPU_STRONG_ORDERED_SHARED | MPU_PRIV_RW_USER_RW }
};
```

The MCU BSP sets property of each region. After the property is set, you can access region only by the property that is set by MCU BSP. For example, if you set S-NOR address property to read-only, you cannot write to S-NOR. To check the setting of MPU, refer to **MPU_Init()** function in "*mcu_bsp/sources/dev.drivers/mpu/tcc805x/mpu.c*". For more detailed information on MPU, refer to ARM web site (<http://infocenter.arm.com>).

8.6 Use Fixed DMA Buffer

To use the DMA buffer in non-cacheable areas according to your application, you should check the size of the DMA buffer first before using it. Then, you can configure the size by modifying the definition in *mpu.c* file as below. By default, in MCU BSP, the size of DMA buffer area is set to 16 KB and the size of SRAM area is set to 512 KB. Refer to the default DMA size of MCU BSP below.

■ Definition for DMA Size

```
mpu_bsp/sources/dev.drivers/mpu/tcc805x/mpu.c

/**** 1. SELECT DMA SIZE - CHOICE ONLY ONE ****/

#define MPU_REGION_DMA    MPU_REGION_16KB

#if ( MPU_REGION_DMA == MPU_REGION_4KB )
    #define MPU_DMA_ADDR  MPU_SRAM0_ADDR + 0x3F000
#elif ( MPU_REGION_DMA == MPU_REGION_8KB )
    #define MPU_DMA_ADDR  MPU_SRAM0_ADDR + 0x3E000
#elif ( MPU_REGION_DMA == MPU_REGION_16KB )
    #define MPU_DMA_ADDR  MPU_SRAM0_ADDR + 0x3C000
#elif ( MPU_REGION_DMA == MPU_REGION_32KB )
    #define MPU_DMA_ADDR  MPU_SRAM0_ADDR + 0x38000
#elif ( MPU_REGION_DMA == MPU_REGION_64KB )
    #define MPU_DMA_ADDR  MPU_SRAM0_ADDR + 0x30000
#endif
```

The MCU BSP sets DMA area to non-cacheable as much as DMA size. If you use Green Hills IDE Environment, you need to modify the predefinition (DMA_NC_SIZE, default value:0x4000) as follows. This predefinition is used in linker script. If you use Green Hills Environment, refer to “Define Memory Layout (Green Hills IDE Environment)” in Chapter 8.3.

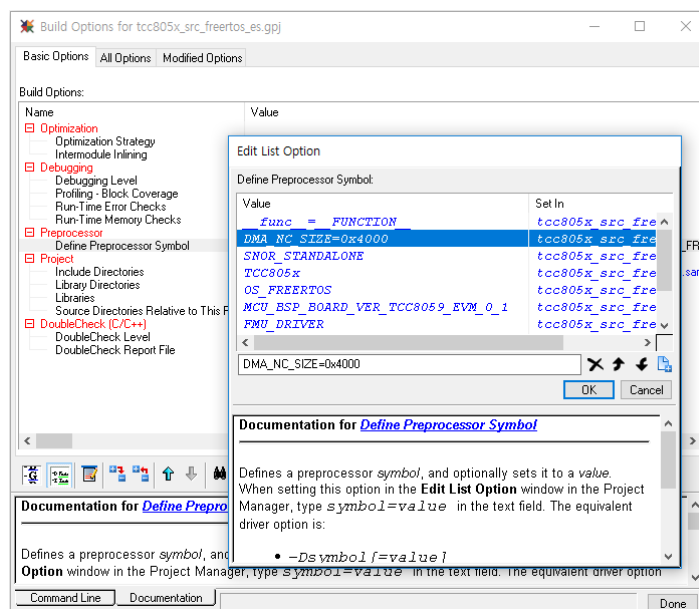


Figure 8.2 Predefinition for DMA (Green Hills IDE Environment)

In case of GCC environment, the size of the DMA buffer can be configured by changing the predefinition (DMA_NC_SIZE, default value:0x4000) for your system. DMA_NC_SIZE is defined in Makefile. If you want to change DMA size, you should change DMA_NC_SIZE in Makefile below. This predefinition is also used in linker script as like Green Hills Environment. If you use GCC Environment, refer to "Define Memory Layout (GCC Environment)" in Chapter 8.3.

■ Pre-definition for DMA (GCC Environment)

mcu_bsp/sources/build/tcc805x/gcc/Makefile

```
COMMON_FLAGS += -O0
COMMON_FLAGS += -finline
COMMON_FLAGS += -fno-builtin
COMMON_FLAGS += -fno-common
COMMON_FLAGS += -fno-gcse
COMMON_FLAGS += -fno-strict-aliasing
COMMON_FLAGS += -marm
COMMON_FLAGS += -mcpu=cortex-r5
COMMON_FLAGS += -nostdinc
COMMON_FLAGS += -g
COMMON_FLAGS += -gdwarf-2
COMMON_FLAGS += -pipe
COMMON_FLAGS += -D__GNU_C__
#COMMON_FLAGS += -DDSE_DATAABORT
#COMMON_FLAGS += -DSUBSYS_USERMODETEST
#COMMON_FLAGS += -DGIC_FIQTEST
COMMON_FLAGS += -DTCC805x
COMMON_FLAGS += -DDMA_NC_SIZE=0x4000
COMMON_FLAGS += -DFMU_DRIVER
COMMON_FLAGS += -DPMIO_DRIVER
```

You can use the DMA buffer address returned from **MPU_GetDMABaseAddress()** function.

■ MPU Function to get DMA Base Address

mcu_bsp/sources/dev.drivers/mpu/tcc805x/mpu.c

```
uint32 MPU_GetDMABaseAddress( void )
{
    uint32 uiDMAStart;

    uiDMAStart = ( uint32 )( &__nc_dmastart );

    return uiDMAStart;
}
```

The **MPU_GetDMABaseAddress()** function returns only the DMA based address. If you want to provide many types of DMA buffer for your application, you need to implement the DMA buffer management depending on use-cases.

Note: The MCU BSP does not limit how this DMA buffer area is partitioned. The use-case and total size of DMA buffer should be strictly classified.

8.7 Detailed View of SRAM

SRAM is categorized into five areas as follows: vector area, data area, zero initialized area, ARM stack area, and Non-cacheable DMA area.

- The vector table is prepared for each ARM exception entry table such as undefine.
- .bss (ZI) size and .data (RW) size vary depending on the function of application.
- The size of the Non-cacheable DMA area is configured by redefining the DMA_NC_SIZE parameter (dma_memory for GHC and NC_SRAM for GCC environment).
- The ARM stack area is reserved stack for each ARM execution mode, and it starts at the end of the Non-cacheable DMA area area.

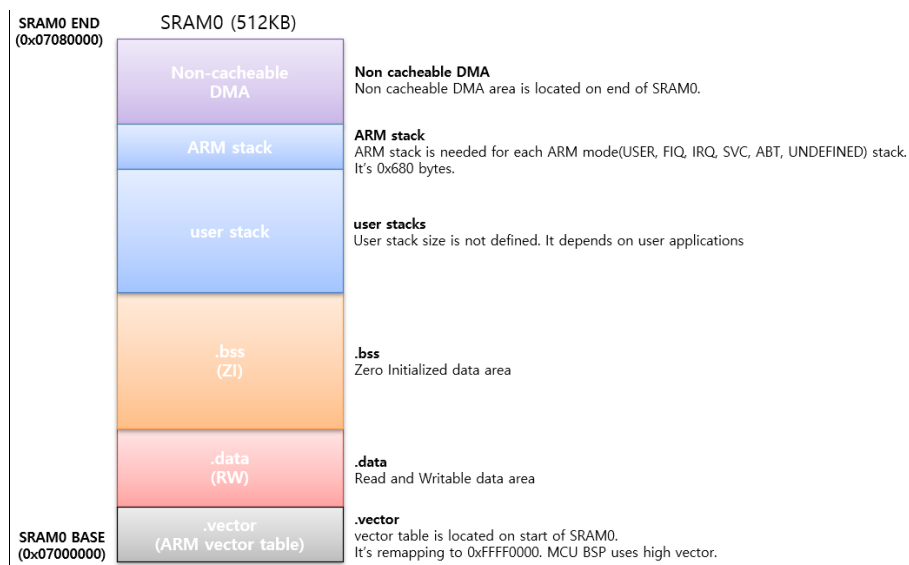


Figure 8.3 SRAM Map

9 MCU BSP RTOS

TCC805x MCU BSP can use Real Time Operating System (RTOS) function via System adaptation Layer (SAL). The SAL originally supports simple RTOS such as FreeRTOS, uC/OS-III, etc. The SAL of MCU BSP currently supports FreeRTOS and uC/OS-III. MCU BSP uses FreeRTOS by default. Figure 9.1 shows the position of the FreeRTOS in MCU BSP. For the uC/OS-III, the uC/OS-III is supportable based on the contract with Micrium [4].

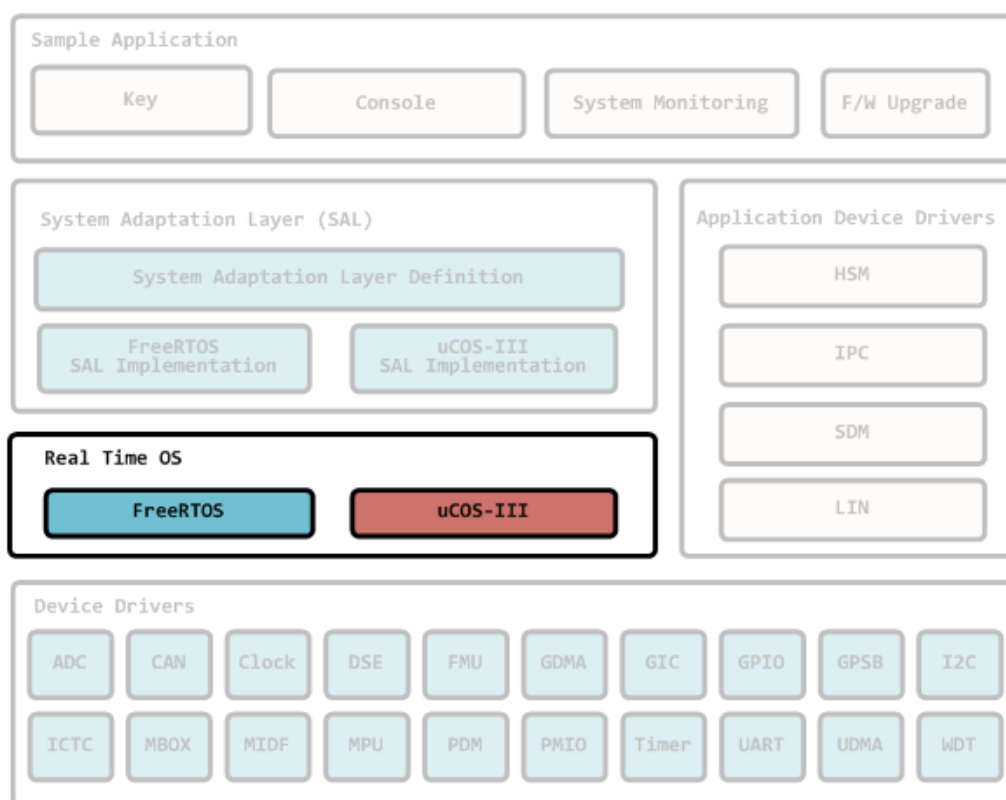


Figure 9.1 RTOS in MCU BSP

The SAL supports simple Operating System (OS) functions such as task, queue, semaphore, and event flags management. Actually, MCU BSP application uses a small part of the whole OS function and is defined in "*sal_com.h*" file. The header file contains prototype of SAL APIs, error value, and some configuration definitions. This chapter describes FreeRTOS because TCC805x MCU BSP uses FreeRTOS by default.

9.1 Initiation Procedure

The following procedure initiates variables and necessary internal tasks for initializing OS.

1. Generate User Start Task: To generate the first task after starting OS, the system calls **Main_StartTask()** function in the "**mcu_bsp/sources/app.sample/app.base/main.c**".
2. Start OS: The MCU BSP RTOS is operated by calling **SAL_OsStart()**. This function sets timer0 for OS tick. OS tick is a periodic interrupt by a timer and enables the OS to execute the priority-based task schedules. Interval of OS Tick is 1 ms. Refer to **vConfigureTickInterrupt()** function in "**mcu_bsp/sources/os/freertos/portable/ARM_CR5**".
3. Generate user application task: You can add a function to generate task for each application. Refer to **AppTaskCreate()** function in "**mcu_bsp/sources/app.sample/app.base/main.c**".

9.2 SAL Interface

SAL interface is required for implementation according to the OS. If you want to use FreeRTOS, the SAL should be implemented as a function supported by FreeRTOS in "**sal_freertos_impl.c**" file. For more detailed information about SAL interface, refer to "**TCC805x MCU BSP-API Specification for System Adaptation Layer**". [2]

9.3 FreeRTOS OS Kernel

The MCU BSP uses FreeRTOS v10.3.1 and you can download the FreeRTOS v10.3.1 from [FreeRTOS download page](#). [3]

FreeRTOS OS Kernel layer contains FreeRTOS kernel codes that should be used via SAL APIs. Telechips does not guarantee any modification of FreeRTOS kernel codes.

9.3.1 FreeRTOS Directory Structure

The FreeRTOS of MCU BSP is structured as follows.

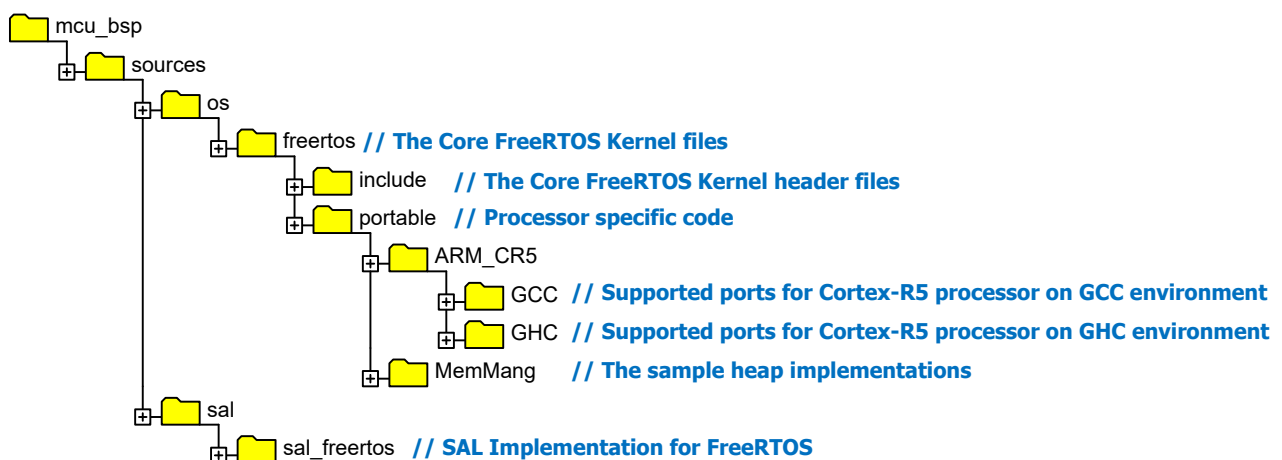


Figure 9.2 Directory Structure of FreeRTOS

9.3.1.1 OS

The “OS” directory contains the OS specific directories as follows:

- freertos: This directory contains the FreeRTOS specific kernel files, which are called as *tasks.c*, *queue.c*, *list.c*, and *event_group.c*. These files are required for FreeRTOS kernel operation.
- include: This directory contains the FreeRTOS kernel header files including *FreeRTOSConfig.h*. You can customize FreeRTOS via *FreeRTOSConfig.h* file.
- portable: This directory contains specific processor and builds environment-related codes. These codes can be modified to use FreeRTOS according to the development environment.
 - ARM_CR5: The TCC805x MCU BSP operates based on Cortex R5 single core. The FreeRTOS supports *port.c* and *portASM.S* files. You can port FreeRTOS by modifying these files.
 - MemMang: The FreeRTOS supports memory allocation implementations which have five sample codes such as heap_1, heap_2, heap_3, heap_4, and heap_5. For more information on memory allocation implementations, refer to the FreeRTOS web page or “*ReadMe*” file in this directory. However, the TCC805x MCU BSP uses MemMang for queue buffer allocation case only. Therefore, you must not use the memory allocation.

9.3.1.2 SAL

The “sal” directory contains the SAL related codes.

- sal_freertos: This directory contains the “*sal_freertos_impl.c*” file. The APIs in this file call FreeRTOS specific kernel function. SAL can use FreeRTOS kernel function via these APIs and you can use the SAL.

For detailed information on SAL, refer to “*TCC805x MCU BSP-API Specification for System Adaptation Layer*”. [2]

9.3.2 Configuration for FreeRTOS

You should configure a Makefile to use FreeRTOS. The following shows configuration of Makefile in GCC environment.

- Makefile Flags for FreeRTOS

```
mcu_bsp/sources/build/tcc805x/gcc/Makefile
```

```
# Build Flags
```

```
MCU_BSP_BUILD_FLAGS_TARGET_OS_FREERTOS    ?= 1
```

```
MCU_BSP_BUILD_FLAGS_TARGET_OS_UCOS        ?= 0
```

```
MCU_BSP_BUILD_FLAGS_TARGET_OS_NONE        ?= 0
```

In Green Hills environment, you can use freeRTOS Green Hills project.

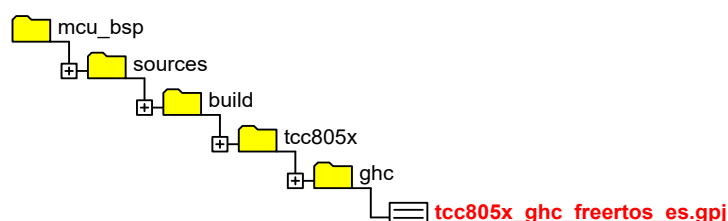


Figure 9.3 Green Hills Project for FreeRTOS

9.3.3 Configuration for Task Priorities

Each task is assigned by a priority from 0 to 'configMAX_PRIORITIES - 1', where configMAX_PRIORITIES is defined in FreeRTOSConfig.h. Default value of configMAX_PRIORITIES is 16.

In case of FreeRTOS, the priority number represents the task priority. For example, if the task priority is 0, it means the lowest task priority. The task priority can be duplicated without any restrictions, and the same number means the same task priority. These task priorities are defined in "*sal_com.h*" file. Each task priority can be adjusted according to your system. The following shows the task priorities of FreeRTOS.

■ RTOS Task Priorities

mcu_bsp/sources/sal/sal_com.h

```
typedef enum _SALTaskPriority_
{
    SAL_PRIO_APP_CFG           = 2,    /**< APP TASK START task          */
    SAL_PRIO_POWER_MANAGER     = 3,    /**< Power Manager APP task        */
    SAL_PRIO_IPC_CONTROL       = 3,    /**< IPC Control task              */
    SAL_PRIO_SYSTEM_MON        = 4,    /**< System Monitoring APP task     */
    SAL_PRIO_CAN_DEMO          = 4,    /**< CAN DEMO APP task             */
    SAL_PRIO_VSDR_MONITOR      = 5,    /**< Vehicle Signal DEMO APP task   */
    SAL_PRIO_VEHICLE_SIGNAL_DEMO = 6,  /**< VSDR Receive Monitor Task     */
    SAL_PRIO_IPC_PARSER        = 6,    /**< IPC Parser task               */
    SAL_PRIO_KEY_APP           = 6,    /**< KEY APP task                  */
    SAL_PRIO_UART_TEST         = 6,    /**< UART Test APP task            */
    SAL_PRIO_CONSOLE_DEMO      = 6,    /**< Console DEMO APP task         */
    SAL_PRIO_NVM_MAMAGER       = 8,    /**< NVM Manager APP task          */
    SAL_PRIO_FWUG_APP          = 10,   /**< FWUG APP task                 */
    SAL_PRIO_LOWEST            = 12,   /**< APP TASK START task          */
} SALTaskPriority_t;
```

9.3.4 Customizing "FreeRTOSConfig.h" File

The FreeRTOS is customized by using configuration file, *FreeRTOSConfig.h*. This file is located in the path *src/OS/freertos/include*. The following shows configured definitions for FreeRTOS.

■ FreeRTOS Configuration File

<i>mcu_bsp/sources/os/freertos/include/FreeRTOSConfig.h</i>	
<code>#define configMAX_API_CALL_INTERRUPT_PRIORITY</code>	<code>0//(16 -2)</code>
<code>#define configCPU_CLOCK_HZ</code>	<code>100000000UL</code>
<code>#define configUSE_PORT_OPTIMISED_TASK_SELECTION</code>	<code>0</code>
<code>#define configUSE_TICKLESS_IDLE</code>	<code>0</code>
<code>#define configTICK_RATE_HZ</code>	<code>((TickType_t) 1000)</code>
<code>#define configPERIPHERAL_CLOCK_HZ</code>	<code>(33333000UL)</code>
<code>#define configUSE_PREEMPTION</code>	<code>1</code>
<code>#define configUSE_IDLE_HOOK</code>	<code>0</code>
<code>#ifdef TCC_FREERTOS_DEMO</code>	
<code>#define configUSE_TICK_HOOK</code>	<code>1 //TCC_DEMO</code>
<code>#else</code>	
<code>#define configUSE_TICK_HOOK</code>	<code>0 //org</code>
<code>#endif</code>	
<code>#define configMAX_PRIORITIES</code>	<code>(16)</code>
<code>#define configMINIMAL_STACK_SIZE</code>	<code>((unsigned short) 250)</code>
<i>/* Large in case configUSE_TASK_FPU_SUPPORT is 2 in which case all tasks have an FPU context. */</i>	
<code>#ifdef TCC_FREERTOS_DEMO</code>	
<code>#define configTOTAL_HEAP_SIZE</code>	<code>(100 * 1024)</code>
<code>#else</code>	
<code>#define configTOTAL_HEAP_SIZE</code>	<code>(1 * 1024)</code>
<code>#endif</code>	
<code>#define configMAX_TASK_NAME_LEN</code>	<code>(30)</code>
<code>#define configUSE_TRACE_FACILITY</code>	<code>0</code>
<code>#define configUSE_16_BIT_TICKS</code>	<code>0</code>
<code>#define configIDLE_SHOULD_YIELD</code>	<code>1</code>
<code>#define configUSE_MUTEXES</code>	<code>1</code>
<code>#define configQUEUE_REGISTRY_SIZE</code>	<code>8</code>
<code>#define configCHECK_FOR_STACK_OVERFLOW</code>	<code>2</code>
<code>#define configUSE_RECURSIVE_MUTEXES</code>	<code>1</code>
<code>#define configUSE_MALLOC_FAILED_HOOK</code>	<code>0</code>
<code>#define configUSE_APPLICATION_TASK_TAG</code>	<code>0</code>
<code>#define configUSE_COUNTING_SEMAPHORES</code>	<code>1</code>
<code>#define configUSE_QUEUE_SETS</code>	<code>1</code>
<code>#define configUSE_TASK_NOTIFICATIONS</code>	<code>1</code>

The FreeRTOS is defined in "*FreeRTOSConfig.h*" file. Refer to the "*Customization section*" on [FreeRTOS web page](#) for more information.

9.3.5 Memory Resource Function

The FreeRTOS uses standard library such as *memset()*, *memcpy()*, *strcpy()*, and so on. Refer to "*mcu_bsp/sources/sal/sal_freertos/sal_freertos_impl.c*" file.

9.3.6 Exception Handler

Exception handlers are defined in "*mcu_bsp/sources/core/GCC/vector.S*" file in case of GCC environment. The handlers below are executed when an exception occurs. The following shows the defined exception handlers.

■ Vector Table for ARM Exception

```
mcu_bsp/sources/core/GCC/vector.S

_vector:
    ldr pc, =reset_handler
    ldr pc, =ARM_ExceptUndefInstrHndlr
    ldr pc, =ARM_ExceptSwiHndlr
    ldr pc, =ARM_ExceptPrefetchAbortHndlr
    ldr pc, =ARM_ExceptDataAbortHndlr
    ldr pc, =arm_reserved
    ldr pc, =ARM_ExceptIrqHndlr
    ldr pc, =ARM_ExceptFiqHndlr
```

The exception handlers are different for each OS. For example, when an undefined exception occurs, *FreeRTOS_ARM_ExceptUndefInstrHndlr()* handler is executed in case of FreeRTOS. The following shows implemented exception handlers.

■ SAL Exception Handler

```
mcu_bsp/sources/sal/sal.S

/*****
 *
 *                      UNDEFINED INSTRUCTION EXCEPTION HANDLER
 *****/

ARM_ExceptUndefInstrHndlr:
#ifdef OS_FREERTOS
    B      FreeRTOS_ARM_ExceptUndefInstrHndlr
#else
    B      OS_CPU_ARM_ExceptUndefInstrHndlr
#endif
```

10 USE TECHNOLOGY BY THE THIRD PARTY WITHIN MCU BSP

Technologies of Telechips may include the technologies of the third party (e.g. uC/OS-III). The third party is responsible for all or any of intellectual property licenses for any special, indirect, incidental, or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

Technologies are provided as they are without any other implied or express warranty. The third party does not provide any implied warranty including suitability or marketability for a specific use.

11 REFERENCES

- [1] TCC805x Full Specification
- [2] Contact Telechips for more details: sales@telechips.com
- [3] <https://www.freertos.org/a00104.html>
- [4] <https://www.micrium.com/>

12 REVISION HISTORY

Rev. 1.02: 2021-04-09

- Updated
 - Chapter 3: Figure 3.1
 - Chapter 8: Figure 8.1
 - Chapter 9: Figure 9.1

Rev. 1.01: 2021-01-22

- Added
 - Chapter 6: Reset Process

Rev. 1.00: 2020-11-12

- First version release

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.

No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.

This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.

Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Copyright in the material provided by Telechips, Inc. is owned by Telechips unless otherwise noted.

For reproduction or use of Telechips' copyright material, permission should be sought from Telechips. That permission, if given, will be subject to conditions that Telechips' name should be included and interest in the material should be acknowledged when the material is reproduced or quoted, either in whole or in part. You must not copy, adapt, publish, distribute or commercialize any contents contained in the material in any manner without the written permission of Telechips. Trade marks used in Telechips' copyright material are the property of Telechips.

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:

"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use other firmware of mp3:

"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use Digital Wave DRA solution:

"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:

"This product made under license to certain U.S. patents and/or foreign counterparts."
"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:

"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:

"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:

"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."